

Compte rendu du 15 juin 2026

July 3, 2026

Liste des Questions possibles

1. Est-ce qu'un argument X est accepté sceptiquement dans une extension sous la sémantique σ ?
2. Est-ce qu'un argument X est accepté crûdement dans une extension sous la sémantique σ ?
3. Est-ce qu'un argument X est rejeté de toutes les extensions sous la sémantique σ ?
4. Énumérer toutes les extensions sous la sémantique σ ?
5. Est-ce que l'extension E est dans au moins une sémantique?

Liste des solutions possibles

1. La solution est de type Boolean.
2. La solution est une liste d'extensions.

Pour chacun des scénarios on supposera que si l'on ne peut pas conclure on fera appeler à un solveur directement.

Scénario 1 : Réduction par la sémantique fondée et Isomorphisme

Le Problème Cible :

On dispose d'un framework d'argumentation (AF) cible F composé de 4 arguments $\{a, b, c, d\}$ et on cherche à répondre à la question sous-jacente : **Quelles sont ses extensions préférées ?**



La Base de Cas :

On possède une base de cas contenant des problèmes résolus. Par exemple, on dispose du cas C_1 dont la situation est $F_1 = \{x \leftrightarrow y\}$ (un dilemme simple). La question porte sur les extensions préférées, et la solution connue est $S_1 = \{\{x\}, \{y\}\}$.

Pour résoudre notre problème cible F sans avoir à explorer toutes les combinaisons, on va chercher à réduire le graphe au maximum afin de rechercher ensuite si le cœur du problème a déjà été étudié dans le passé.

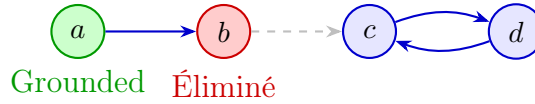
Étape 1 : Calcul de la Grounded

On commence par identifier les arguments inconditionnellement acceptés. Ici, l'argument a n'a aucun attaquant, il est donc obligatoirement vrai.

$Grounded = \{a\}$.

Étape 2 : Obtention du graphe réduit (*reduc*)

On obtient le graphe réduit en propageant cette certitude : on supprime les arguments de la sémantique fondée, les arguments qu'ils attaquent (qui sont définitivement éliminés), et toutes les attaques associées. Ici, puisque a est accepté, il élimine définitivement b . L'attaque de b vers c devient inactive et disparaît. Il reste le graphe réduit $F_{reduc} = \{c, d\}$ avec l'attaque mutuelle $c \leftrightarrow d$.



Étape 3 : Recherche d'isomorphisme dans la base

Si le graphe réduit est non vide, on recherche si on trouve un cas similaire à un isomorphisme près dans notre base. Notre graphe réduit F_{reduc} est un simple cycle $c \leftrightarrow d$. En interrogeant la base, on trouve une correspondance structurelle parfaite avec le cas C_1 ($x \leftrightarrow y$).

Étape 4 : Bijection et Conclusion

On renomme les arguments sous une bijection pour calquer la solution extraite : $x \mapsto c$ et $y \mapsto d$. La solution locale de C_1 se traduit alors par : $\{\{c\}, \{d\}\}$.

Pour obtenir la solution globale de notre graphe complet F , il suffit de fusionner la sémantique fondée (le contexte certain) avec les extensions locales issues du RaPC.

Extensions préférées = $\{a\} \cup \{c\}$ et $\{a\} \cup \{d\}$.

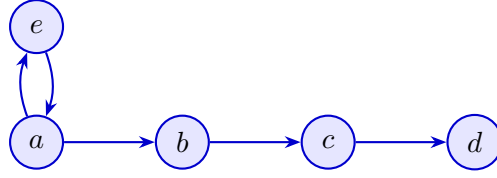
Solution globale : $\{\{a, c\}, \{a, d\}\}$.

Ce nouveau cas a pu nous apporter des informations supplémentaires donc on l'ajoute à la base de cas

Scénario 2 : Échec de la Grounded et Réduction par Motif Topologique

Le Problème Cible :

On dispose d'un framework d'argumentation (AF) cible F composé de 5 arguments $\{a, b, c, d, e\}$. La question sous-jacente est : **Quelles sont ses extensions préférées ?**



La Base de Cas :

On possède une base de cas contenant des problèmes résolus. Par exemple, le cas C_2 dont la situation est $F_2 = \{x \leftrightarrow y, y \rightarrow z\}$ (un dilemme attaquant un singleton). Sa question porte sur les extensions préférées et la solution est $S_2 = \{\{x, z\}, \{y\}\}$.

Étape 1 : Calcul de la Grounded

On cherche d'abord à calculer la sémantique fondée pour réduire le graphe. Cependant, dans ce graphe, **aucun argument n'est inattaqué**. Le cycle $a \leftrightarrow e$ empêche d'avoir une source claire.

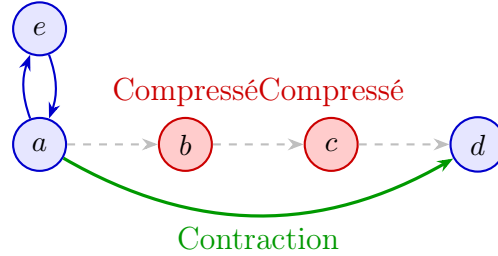
$Grounded = \emptyset$. La sémantique fondée ne nous apporte aucune information pour nettoyer le graphe.

Étape 2 : Identification et Réduction de motifs

On cherche à isoler des **motifs topologiques** connus que l'on peut réduire facilement sans altérer la sémantique globale.

On remarque le sous-motif linéaire $a \rightarrow b \rightarrow c \rightarrow d$.

Dans la théorie de l'argumentation, une chaîne de longueur impaire (3 attaques : $a \rightarrow b \rightarrow c \rightarrow d$) se comporte sémantiquement comme une attaque directe. En effet, a attaque b , donc a défend c , et par conséquent a attaque indirectement d . La chaîne se compresse en $a \rightarrow d$.



On réduit le motif : la structure $a \leftrightarrow e, a \rightarrow b \rightarrow c \rightarrow d$ est compressée et devient l'équivalent sémantique de $a \leftrightarrow e, a \rightarrow d$. Le graphe réduit est $F_{reduc} = \{a, e, d\}$.

Étape 3 : Recherche d'isomorphisme dans la base

On interroge notre base de cas avec ce nouveau graphe réduit F_{reduc} . On trouve une correspondance parfaite à un isomorphisme près avec le cas C_2 ($x \leftrightarrow y, y \rightarrow z$).

Étape 4 : Bijection et Conclusion

On renomme les arguments sous la bijection suivante : $x \mapsto e, y \mapsto a, z \mapsto d$.

La solution de C_2 était $S_2 = \{\{x, z\}, \{y\}\}$. En appliquant la bijection, la solution locale devient $\{\{e, d\}, \{a\}\}$.

Puisque nous avons compressé le milieu de la chaîne (b et c), il suffit de réintégrer ces arguments selon la logique de polarité :

- Si a est accepté : b est éliminé, c est accepté (et d est bien éliminé par c). Cela donne la branche $\{a, c\}$.
- Si e est accepté : a est éliminé, b est accepté, c est éliminé (et d est bien sauvé). Cela donne la branche $\{e, b, d\}$.

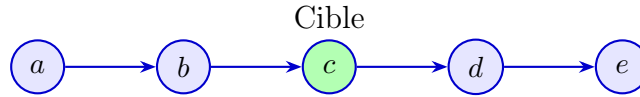
Solution globale : $\{\{e, b, d\}, \{a, c\}\}$.

Scénario 3 : Réduction Directionnelle (Élagage des Successeurs)

Verdict final pour l'argument cible : L'argument c est **Accepté**.

Le Problème Cible :

On dispose d'un framework d'argumentation (AF) cible F composé de 5 arguments formant une chaîne : $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$. La question sous-jacente est locale : **Est-ce que l'argument c appartient à une extension préférée ?**

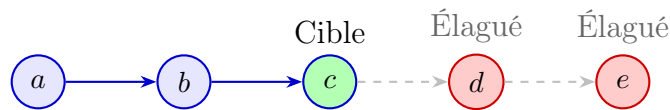


La Base de Cas :

On possède le cas de référence C_3 avec la situation $F_3 = \{x \rightarrow y \rightarrow z\}$ et la question "Est-ce que z est accepté sous la sémantique préférée?". La solution extraite est $S_3 = \text{True}$.

Étape 1 : Élagage directionnel

Traitement standard appliqué. Le parcours des prédécesseurs isole le sous-graphe pertinent en écartant les avaloirs non-cycliques (d et e).



Le graphe réduit est formellement acté à $F_{\text{reduc}} = \{a, b, c\}$.

Étape 2 : Mapping RaPC

Traitement standard appliqué. L'isomorphisme est validé avec le cas de référence C_3 .

Étape 3 : Bijection et Conclusion

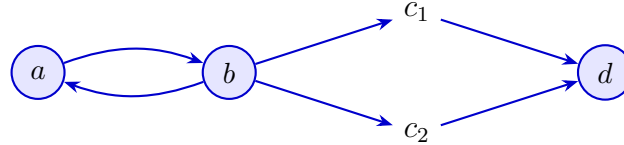
Transformation finale directe. L'application de la bijection ($x \mapsto a, y \mapsto b, z \mapsto c$) sur la solution S_3 garantit le transfert du statut d'acceptation de manière stricte et définitive.

Scénario 4 : Réduction par Fusion de Clones (Arguments Symétriques)

Solution globale (Extensions préférées) : $\{\{a, c_1, c_2\}, \{b, d\}\}$

Le Problème Cible :

Framework d'argumentation (AF) cible $F = \{a, b, c_1, c_2, d\}$ structuré par un dilemme initial $a \leftrightarrow b$ attaquant deux arguments parallèles.



La Base de Cas :

Cas de référence C_4 disponible avec la situation $F_4 = \{x \leftrightarrow y \rightarrow z \rightarrow w\}$ et la solution extraite $S_4 = \{\{x, z\}, \{y, w\}\}$.

Étape 1 : Initialisation

Les conditions initiales sont validées. La sémantique fondée ($Grounded = \emptyset$) ne déclenche pas de réduction déterministe.

Étape 2 : Fusion des clones

Traitement standard appliqué. L'analyse topologique identifie formellement la symétrie parfaite entre c_1 et c_2 (mêmes attaquants en amont, mêmes cibles en aval). Le graphe subit une compression de fusion, condensant les deux nœuds en un méta-argument C_{12} .



Étape 3 : Mapping RaPC

L'isomorphisme est formellement établi avec le cas C_4 de la base ($a \leftrightarrow b \rightarrow C_{12} \rightarrow d \equiv x \leftrightarrow y \rightarrow z \rightarrow w$).

Étape 4 : Décompression et Conclusion

Transformation finale directe. L'application de la solution de C_4 sur le graphe réduit dicte la restitution. Le méta-argument C_{12} est décompressé, forçant la duplication du statut d'acceptation pour c_1 et c_2 dans toutes les branches de l'arbre des probabilités.

Scénario 5 : Résolution par Similarité Structurale et Transformation

Contexte Initial :

On dispose d'un framework d'argumentation (AF) cible et d'une base de cas. Cette base

contient un historique de cas composés chacun d'un problème (un graphe et une question) et d'une solution répondant à la question sous-jacente au problème.

Contrairement aux scénarios précédents, on supposera ici qu'aucune méthode de réduction préalable (Grounded, motifs topologiques ou élagage) n'a permis de faire ressortir un isomorphisme parfait avec la base de cas. On procède donc à une approche d'adaptation par similarité.

Étape 1 : Évaluation de la similarité

Le système compare le graphe cible à l'ensemble des cas connus dans la base de cas. Plutôt que de chercher une correspondance stricte, l'algorithme calcule et attribue un **score de similarité** à chacun des cas en fonction du chevauchement de leur topologie (nombre d'arguments communs, structures de cycles similaires, chaînes d'attaques identiques, question identique etc...).

Étape 2 : Sélection et observation des écarts

On conserve uniquement le cas de référence ayant obtenu le **score de similarité maximum**. L'algorithme observe et isole alors formellement les différences structurelles exactes entre le graphe cible et ce cas de référence (ex: une attaque supplémentaire, une direction inversée).

Étape 3 : Évaluation de la réductibilité

Le système analyse ces différences pour déterminer si l'on est capable de "retrouver" le graphe du cas de référence à partir de notre graphe cible à l'aide de traitement implémenter (voir papier Incremental Computation in Dynamic Argumentation Frameworks et Changement dans un système d'argumentation: suppression d'un argument, Change in Abstract Argumentation Frameworks: Adding an Argument).

Étape 4 : Application des transformations

Si le graphe cible peut être ramené au cas de référence, on applique différentes **méthodes de transformation**. Le système adapte la solution du cas extrait en calculant l'impact local des différences observées.

Étape 5 : Conclusion

La solution ainsi transformée et ajustée est validée comme étant la réponse exacte au graphe cible. Le problème est résolu par adaptation analogique, et ce nouveau cas hybride pourra éventuellement être ajouté à la base pour enrichir les futures recherches par isomorphisme.

Scénario 6 : La décomposition en Composantes fortement connexes

Ce scénario s'avère plus complexe, car son lien avec la base de cas n'est pas direct. En effet, la résolution locale des sous-problèmes, caractérisés par des composantes fortement connexes (CFC), ne garantit pas une reconstruction aisée de la solution globale. L'impact des arcs entrants d'une CFC dépend de l'acceptabilité de l'argument source. Ainsi, pour

s'assurer de pouvoir reconstruire le problème initial, il faudrait stocker 2^n graphes différents pour chaque cas, ce qui correspond à l'ensemble des combinaisons d'entrées possibles. Néanmoins, la décomposition en CFC demeure pertinente si l'on emploie un solveur indépendant de la base de cas.

Pour la suite l'idée est de combiné ces scénarios afin d'affiner le problème.

Extraction des Spécialistes

De ces scénarios on peut extraire un certain nombre de spécialistes:

1. Un spécialiste qui permet de rechercher dans la base de cas s'il existe un cas isomorphe à notre cas source : **IsomorphismSpecialist**
2. Un spécialiste qui permet de calculer la sémantique fondée d'un graphe d'argumentation : **GroundedSpecialist**
3. Un spécialiste qui permet de calculer le graphe réduit d'un graphe d'argumentation sous la sémantique fondée : **GroundedReductionSpecialist**
4. Un spécialiste qui permet de rechercher des motifs que nous sommes capables de réduire : **PatternIdentificationSpecialist**
5. Un spécialiste pour réduire des motifs connus (abstrait) : **TopologicalPatternSpecialist**
 - (a) Un spécialiste pour réduire les chaînes linéaires : **LinearPatternSpecialist**
6. Un spécialiste pour transformer un graphe d'argumentation (abstrait) : **TransformationSpecialist**
 - (a) Un spécialiste pour ajouter une attaque : **AddAttackTransformationSpecialist**
 - (b) Un spécialiste pour ajouter un argument : **AddArgumentTransformationSpecialist**
 - (c) Un spécialiste pour retirer une attaque : **RemoveAttackTransformationSpecialist**
 - (d) Un spécialiste pour retirer un argument : **RemoveArgumentTransformationSpecialist**
7. Un spécialiste pour identifier des clones : **CloneIdentifySpecialist**
8. Un spécialiste pour fusionner les clones : **CloneFusionSpecialist**
9. Un spécialiste pour recomposer une solution (abstrait) : **SolutionAdaptationSpecialist**
 - (a) Un spécialiste pour réintégrer les arguments qui ont été fusionnés : **DecompressionSpecialist**
 - (b) Un spécialiste pour fusionner une solution réduite avec la fondée : **GroundedMergeSolutionSpecialist**
10. Un spécialiste pour calculer l'impact d'un changement sur la solution : **TransformImpactAdaptationSpecialist**
11. Un spécialiste pour calculer la similarité des cas dans la base de cas : **SimilaritySpecialist**
12. Un spécialiste pour élaguer les successeurs d'un argument qui nous intéresse : **DirectionalPruningSpecialist**
13. Un spécialiste pour décomposer un graphe d'argumentation en composantes fortement connexes : **SCCDecompositionSpecialist**

14. Un spécialiste qui permet de résoudre le problème en cas de besoin si aucune méthode n'a marché : **DirectResolutionSpecialist**
15. Un spécialiste pour vérifier si un spécialiste est applicable ou non : **ApplicabilityCheckSpecialist**
16. Un spécialiste pour faire la bijection entre les nœuds si l'on a deux isomorphes : **BijectionSpecialist**
17. Un spécialiste pour calculer l'écart entre le cas le plus similaire et le cas source : **DeltaExtractionSpecialist**
18. Un spécialiste pour analyser la question posée par l'utilisateur et orienter la stratégie (crédule, sceptique, etc.) : **QuestionAnalysisSpecialist**
19. Un spécialiste chargé de consolider et d'ajouter un nouveau cas résolu dans la base de cas pour un apprentissage incrémental : **BaseEnrichmentSpecialist**

Liste des Stratégies

1. **GroundedOnlyStrategy** : Stratégie appliquée lorsque la sémantique fondée suffit à résoudre entièrement le problème, c'est-à-dire lorsque le graphe réduit est vide après le calcul de la fondée. Elle ne nécessite pas de base de cas.
 - (a) Calcul de la sémantique fondée.
 - (b) Calcul du graphe réduit.
 - (c) Si le graphe réduit est vide, la solution est directement déduite de la fondée.
2. **SCCStrategy** : Stratégie basée sur la décomposition du graphe d'argumentation en composantes fortement connexes (SCC). Chaque composante est résolue indépendamment, puis les solutions sont fusionnées. Elle ne nécessite pas de base de cas.
 - (a) Décomposition du graphe en composantes fortement connexes.
 - (b) Résolution de chaque composante indépendamment.
 - (c) Fusion des solutions partielles en une solution globale.
3. **DirectResolutionStrategy** : Stratégie de dernier recours. Aucune méthode de réduction n'ayant fonctionné, le problème est transmis directement à un solveur externe.
 - (a) Vérification que toutes les autres stratégies ont échoué ou sont inapplicables.
 - (b) Appel au spécialiste de résolution directe.
4. **GroundedIsomorphismStrategy** (CBR) : Stratégie correspondant au scénario 1. Elle exploite la sémantique fondée pour réduire le graphe, puis recherche un cas isomorphe dans la base de cas afin d'en extraire et d'adapter la solution.
 - (a) Calcul de la sémantique fondée.
 - (b) Calcul du graphe réduit.
 - (c) Recherche d'un cas isomorphe au graphe réduit dans la base de cas.

- (d) Application de la bijection entre les nœuds du cas source et du cas cible.
 - (e) Fusion de la solution issue du cas avec le contexte certain fourni par la fondée.
5. **TopologicalIsomorphismStrategy** (CBR) : Stratégie correspondant au scénario 2. Lorsque la sémantique fondée ne permet pas de réduire le graphe, elle identifie et compresse des motifs topologiques connus avant de rechercher un isomorphisme dans la base de cas.
- (a) Calcul de la sémantique fondée (résultat vide).
 - (b) Détection de motifs topologiques réductibles dans le graphe.
 - (c) Réduction des motifs détectés.
 - (d) Recherche d'un cas isomorphe au graphe réduit dans la base de cas.
 - (e) Application de la bijection entre les nœuds du cas source et du cas cible.
 - (f) Réintégration des arguments compressés dans la solution globale.
6. **DirectionalCBRStrategy** (CBR) : Stratégie correspondant au scénario 3. Pour une question locale portant sur un argument précis, elle élague les successeurs non pertinents afin de réduire le graphe, puis recherche un cas isomorphe dans la base de cas.
- (a) Élagage des successeurs non cycliques de l'argument cible.
 - (b) Recherche d'un cas isomorphe au graphe élagué dans la base de cas.
 - (c) Application de la bijection entre les nœuds du cas source et du cas cible.
7. **CloneFusionCBRStrategy** (CBR) : Stratégie correspondant au scénario 4. Elle identifie les arguments symétriques (clones) dans le graphe, les fusionne en un méta-argument, puis recherche un cas isomorphe dans la base de cas avant de décompresser la solution.
- (a) Identification des arguments clones dans le graphe.
 - (b) Fusion des clones en un méta-argument.
 - (c) Recherche d'un cas isomorphe au graphe compressé dans la base de cas.
 - (d) Application de la bijection entre les nœuds du cas source et du cas cible.
 - (e) Réintégration des arguments fusionnés dans la solution globale.
8. **SimilarityAdaptationStrategy** (CBR) : Stratégie correspondant au scénario 5. Lorsqu'aucun isomorphisme exact n'est trouvé, elle sélectionne le cas le plus similaire dans la base de cas, calcule l'écart structurel entre ce cas et le problème cible, puis adapte la solution par transformation incrémentale.
- (a) Calcul du score de similarité entre le problème cible et chaque cas de la base.
 - (b) Sélection du cas ayant le score de similarité maximal.
 - (c) Calcul de l'écart structurel entre le cas sélectionné et le problème cible.
 - (d) Vérification de l'applicabilité des transformations nécessaires.
 - (e) Application des transformations pour ramener le cas sélectionné au problème cible.

- (f) Calcul de l'impact des transformations sur la solution du cas source.
- (g) Production de la solution adaptée.